Sustainable Agentic AI

Building Efficient, Scalable, and Sustainable AI

By Navveen Balani (https://navveenbalani.dev/)

The Case for Green Agentic AI

As organizations accelerate their exploration of Generative AI and Agentic AI—systems where autonomous AI agents handle complex, multi-step tasks—there is a critical opportunity to embed efficiency and sustainability from the outset.

The choices made **early in the design and prototyping phases** will shape the long-term **cost, scalability, and environmental impact** of AI systems. Decisions around how agents are orchestrated, how models are selected, how data is processed, and how memory is managed become deeply embedded—often expensive and difficult to reverse once systems scale.

Getting it right early isn't just about smart engineering—it's a **necessary shift in mindset**. Just as security, privacy, and accessibility have become standard considerations in modern software development, cost-efficiency and environmental impact must become habitual in the AI lifecycle—starting from day one.

This matters now because most organizations are still in the early stages of adoption. Prototyping is underway, pilot programs are expanding, and AI maturity is uneven across industries. This is a narrow but crucial window to design AI systems that avoid the inefficiencies, overspending, and environmental impact that plagued previous technology waves—where performance and functionality came first, and sustainability was an afterthought.

At the same time, the **urgency is real and measurable**. Multiple studies consistently highlight the environmental and economic pressures tied to AI and digital infrastructure:

- AI and large-scale computing are already driving sharp increases in **data center energy consumption**.
- Power demand from Generative AI alone could reach 224 TWh by 2027—equivalent to the electricity consumption of some nations.
- The broader ICT sector could contribute up to 14% of global carbon emissions by 2040 if current trends continue.

While the exact numbers vary across reports, the direction is consistent: without change, the digital and AI-driven world is on an unsustainable path unless we rethink how these systems are built and scaled.

This white paper focuses on the opportunity ahead: designing and implementing Green Agentic AI—systems that combine intelligence, autonomy, cost-efficiency, and environmental responsibility.

In the sections that follow, we will:

- Explore the core concepts of Agentic AI and why they matter.
- Present a practical framework for applying lean principles across the full agentic lifecycle.
- Highlight supporting technical practices, including energy-efficient models, optimized training, carbon-aware execution, and cost-efficient inference.
- Introduce the Software Carbon Intensity (SCI) standard to help measure, manage, and reduce AI carbon impact.

By embedding these practices early, organizations can build AI systems that are smarter, more scalable, cost-effective, and environmentally sustainable by design—setting a new benchmark for responsible AI development in the years ahead.

The Rising Impact of AI: Cost and Carbon Concerns

AI is rapidly becoming central to how organizations operate, innovate, and deliver value. From Generative AI models that drive content creation and automation to Agentic AI systems that execute complex, multi-step tasks autonomously, the pace of adoption is accelerating across industries.

This surge, however, brings with it a dual challenge: rising operational costs and growing environmental impact.

AI workloads—whether training, fine-tuning, or inference—consume significant computational power. This translates into:

- **Higher energy consumption** across data centers and cloud infrastructure.
- **Increased carbon emissions**, particularly when workloads are powered by non-renewable energy sources.
- **Escalating costs**, as compute-heavy models drive up cloud spending and infrastructure requirements.

While estimates vary, multiple independent studies converge on one message: the environmental and financial footprint of AI is growing at an unsustainable rate.

This is not just an environmental concern—it is a **business risk**. Organizations that ignore the cost and carbon dimensions of AI systems face mounting challenges:

- **Runaway cloud costs** that erode the business case for AI at scale.

- **Regulatory and stakeholder pressure** to demonstrate environmental responsibility.
- **Inflexibility in AI operations**, as inefficient systems become harder to adapt or optimize.

The path forward is clear: AI systems must be designed for efficiency, scalability, and sustainability from the ground up.

This paper introduces the concept of **Green Agentic AI**—an approach that embeds cost-efficiency and carbon-awareness into every layer of AI development, from model selection and agent orchestration to deployment and measurement.

In the next section, we explore what **Agentic AI** is, how it works, and where the hidden costs and inefficiencies can be addressed early for long-term impact.

What is Agentic AI?

Agentic AI refers to an emerging approach where artificial intelligence systems are built as autonomous, goal-driven agents—or a coordinated society of agents—that can manage and execute complex tasks end-to-end with minimal human intervention.

Unlike traditional AI systems, which often rely on a single monolithic model to respond to inputs, Agentic AI combines multiple specialized components—agents, models, tools, and memory stores—into an orchestrated workflow. You give the system a goal, and it figures out the necessary steps, executes them, reflects on outcomes, and adjusts along the way.

This approach is rapidly gaining traction as AI moves toward more autonomous, self-directed behaviors capable of handling multi-step, multi-system tasks that go beyond one-shot predictions.

Key Characteristics of Agentic AI:

- **Autonomous by Design:**
  Agentic AI systems can take a high-level user goal or request and manage the entire process to achieve it. They break down the problem into subtasks, select the right methods and tools, and carry out the work with minimal or no human intervention. The human's role is to set the goal and constraints, while the AI handles the "heavy lifting" of reasoning, action, and delivery.

- **Reflective and Iterative:**
  Unlike traditional models that generate a single static output, Agentic AI systems are self-reflective. They can evaluate their own results, detect when outcomes fall short of constraints or expectations, and adjust accordingly—rerunning certain steps, selecting alternative models, or refining outputs. This built-in feedback loop makes them more robust, resilient, and adaptable in handling complex, real-world tasks.

- **Orchestration-Centric:**
  Agentic AI is inherently about coordination. Multiple agents, models, APIs, and memory layers work together, each fulfilling specialized roles within an overarching orchestration framework. The system may dynamically call different models or spawn new agents depending on the task context, making it modular, flexible, and scalable—more like a living AI workflow than a static system.

How Agentic AI Works in Practice

To illustrate Agentic AI in action, consider a financial advisory firm using AI to rebalance a client's investment portfolio based on Environmental, Social, and Governance (ESG) criteria—a task that combines financial analysis, regulatory compliance, and personalized client reporting.

A traditional AI approach might attempt to train one large model to take the client's data and produce recommendations directly.

In contrast, an Agentic AI approach breaks the task into distinct stages:

1. **Goal Ingestion:**
   The system receives the high-level instruction: *"Rebalance ESG portfolio for client 84721."*
   A lightweight agent interprets the goal, extracts key details (client ID, ESG focus, portfolio type), and identifies any constraints (e.g., "no more than 5% deviation from current allocations").

2. **Planning & Reasoning:**
   The AI then plans the necessary steps—fetching current portfolio data, applying ESG filters, optimizing asset allocations, assessing financial risk, and generating recommendations.
   This planning may use an orchestration agent or a language model to map out a logical, executable workflow.

3. Tool & Model Selection:
   For each subtask, the system chooses the appropriate tools or models—retrieving data via APIs, using a dedicated ESG scoring model, running risk assessments, and selecting a language model for client communication. This is where the AI decides how to execute each step.

4. **Action Execution:**
   The selected agents and models **perform the actual operations**:

   - ESG scores are applied to holdings.
   - Non-compliant assets are flagged.
   - A new portfolio is calculated that balances ESG improvements with financial targets.
   - A client-friendly report is generated.

5. **Reflection & Retry:**
   The system evaluates the outcome against constraints (e.g., risk limits, sector exposure). If any conditions are not met, it can adjust parameters and retry specific steps—without redoing the entire process.
   The AI may also simplify the language of the generated report if it's too technical, reflecting on feedback or confidence thresholds.

6. **Memory & Learning:**
   The system stores key learnings:

   - Accepted recommendations.
   - Client preferences (e.g., “exclude fossil fuels” or “prefer renewable energy”).
   - Past plan performance or model effectiveness.

   This memory allows the system to personalize future rebalancing without starting from scratch, improving both speed and relevance over time.

The Hidden Risks of Agentic AI

While Agentic AI unlocks new levels of automation and decision-making, it also introduces new risks—particularly when it comes to cost, carbon, and compute efficiency:

- **Unclear goal definitions** that trigger unnecessary work, over-processing, and inflated resource use.
- **Over-planning and over-decisioning**, where agents generate excessive steps, retries, or iterations beyond what is actually required
- **Inefficient tool and model choices**, often defaulting to large, high-cost models without optimization for the task at hand.
- **Execution inefficiencies**, such as redundant actions, verbose outputs, and underutilized infrastructure
- **Uncontrolled reflection and retries** that silently increase compute without proportional value.
- **Redundant memory storage** that bloats context and leads to inefficient downstream execution.

At scale, these inefficiencies can drive up cloud costs, energy consumption, and carbon emissions, undermining the business case for AI while contributing to environmental harm.

The Case for Lean and Green Agentic AI

This is where the concept of Lean and Green Agentic AI comes in. By embedding efficiency principles at every stage—from the initial goal to execution and learning—organizations can:

- **Control costs** by eliminating unnecessary compute and infrastructure waste.
- **Reduce carbon emissions** by optimizing how and when models run.
- **Enhance scalability** by keeping AI systems modular, efficient, and easy to maintain.

Agentic AI done well isn't just intelligent—it's resource-aware, cost-effective, and sustainable by design.

In the next section, we present a practical framework that applies lean thinking systematically across the Agentic AI lifecycle—helping organizations unlock the benefits of automation without the hidden costs.

Designing Lean and Green Agentic AI

Agentic AI systems unlock powerful new capabilities—but without intentional design, they also risk becoming computationally bloated, energy-intensive, and financially unsustainable.

To avoid these outcomes, we introduce a Lean and Green Lifecycle Framework—a practical, stage-by-stage approach to designing Agentic AI systems that are:

- **Cost-efficient** by minimizing redundant processing, model overuse, and infrastructure waste.
- **Environmentally sustainable** by reducing unnecessary compute, energy consumption, and carbon emissions.
- **Operationally scalable** by building in smart constraints and purposeful orchestration from the start.

This framework breaks the Agentic AI lifecycle into **six distinct stages**, each with its own **risks, opportunities, and optimization strategies**:

1. Goal Ingestion
2. Planning & Reasoning
3. Tool & Model Selection
4. Action Execution
5. Reflection & Retry

6. Memory & Learning

Each stage is an opportunity to embed intelligence not just in the agents, but in the way they **consume resources**, **interact**, and **improve over time**.

The following sections walk through each stage in detail, highlighting:

- What happens at that stage,
- Where inefficiencies or emission hotspots may arise, and
- How to apply **lean and green principles** to address them—reducing both cost and carbon.

Stage 1: Goal Ingestion

Goal Ingestion is the first and foundational stage of any Agentic AI system. This is where the system receives a high-level request—from a human user or another digital system—and must transform it into a clear, actionable internal objective that can drive downstream planning and execution.

For example, a simple user instruction such as:
"Rebalance ESG portfolio for client 84721"
needs to be parsed into structured, machine-readable elements:

- The client ID (84721),
- The task type (portfolio rebalancing),
- The constraints (e.g., adherence to Environmental, Social, and Governance [ESG] criteria, risk tolerance, deviation thresholds).

At its core, Goal Ingestion is the system's effort to answer:
"What exactly am I being asked to do, and how can I define this goal internally for efficient execution?"

Depending on the complexity of the request, this stage may involve:

- Natural language understanding to extract intent,
- Entity extraction to identify key inputs,
- Routing logic to pass control to the most appropriate downstream sub-agent or workflow.

For instance, a portfolio rebalancing request concerning equity assets may need to be routed differently from one concerning bond portfolios or multi-asset allocations.

Inefficiencies to Watch Out For

When Goal Ingestion is not designed carefully, it can become the first source of waste, unnecessary complexity, and environmental impact—long before any meaningful computation has taken place.

Several common pitfalls include:

1. Ambiguous Prompts Fed to Oversized Models

Allowing vague or loosely defined requests to flow directly into large-scale language models (LLMs), such as those with 70 billion parameters, can result in:

- Unfocused, verbose, or irrelevant plans,
- Excessive compute time and cost,
- Wasted energy on unproductive iterations.

These oversized models may end up “hallucinating” or over-engineering responses when a simple classification or structured interpretation would suffice.

2. Context Overload

In an effort to avoid missing anything, poorly designed systems often retrieve excessive background information “just in case,” such as:

- The client’s entire transaction and portfolio history,
- All previous interactions or meeting transcripts,
- Complete market data snapshots.

Most of this data is rarely necessary for the immediate task but significantly increases memory consumption, processing time, and emissions.

3. Premature Engagement of Heavyweight Reasoning

Some systems default to complex orchestration or multi-agent planning too early, when simpler, rule-based pathways or low-effort parsing would have been sufficient to triage and route the request.

This can trigger a cascading sequence of unnecessary computations—each consuming compute resources and energy that could have been saved with better early-stage discipline.

Lean and Green Principles Applied

To mitigate these risks, Lean and Green Goal Ingestion focuses on:

- **Clarity**: Ensuring that the user’s intent is precisely understood,
- **Efficiency**: Applying the lightest computational effort needed to interpret the goal, and
- **Minimized context and memory use**: Loading only what is truly necessary.

Key practices include:

1. Improve Prompt Clarity

Whenever possible, use structured input templates or guided user prompts to capture the request. For example:

Instead of:
“Make portfolio greener and smarter.”

The system should guide the user toward:
“Rebalance portfolio 84721 with ESG focus: yes; asset class: equity; maximum deviation: 5%.”

Even when natural language is used, the system can enforce internal prompt parsing standards to extract:

- Entity fields (client ID, asset class),
- Objective (rebalance),
- Constraints or preferences (ESG compliance, allowable deviation).

This upfront clarity reduces downstream uncertainty, making the system more efficient from the first interaction.

2. Use Lightweight Models for Goal Interpretation

Rather than reaching for a large general-purpose LLM to parse every incoming request, a lean design uses:

- Intent classification models (often under 100 million parameters),
- Rules-based decision trees or templates,
- Or fine-tuned small language models purpose-built for command understanding.

For instance, an intent classifier can easily identify that a prompt relates to portfolio rebalancing with ESG criteria—without the need for any heavy compute.

Large models should be reserved for later stages when deeper reasoning or content generation is actually needed.

This approach delivers:

- Lower latency,
- Lower energy consumption, and
- Lower cloud costs.

3. Limit Context to Only Relevant Facts

The system should retrieve and pass forward only minimal, task-relevant context, such as:

- The latest portfolio snapshot,
- The last rebalancing summary, and
- The client's stored ESG preferences.

This avoids pulling in:

- Entire transaction records,
- Full market feeds,
- Legacy documents or irrelevant user history.

Keeping context lean helps:

- Minimize the size of downstream prompts (especially important when calling LLMs with token limits),
- Reduce memory footprint,
- Improve overall system efficiency.

Practical Illustration: Lean vs. Non-Lean Goal Ingestion

Non-Lean Agentic AI	Lean and Green Agentic AI
Receives ambiguous prompt: *"Make the portfolio greener and smarter."*	Seeks clarification or structured input: *"Rebalance portfolio 84721 with ESG focus and a 5% deviation limit."*
Invokes large LLM to infer meaning and plan steps.	Uses lightweight intent classifier or rule-based parsing.
Loads entire transaction history, full market data, and previous reports "just in case."	Loads only the most recent portfolio state and specific client ESG preferences.
Engages heavy planning early, even for simple tasks.	Routes efficiently, deferring complex planning until necessary.

The Lean and Green approach:

- Delivers faster time-to-action,
- Consumes fewer resources,

- Results in **lower costs and reduced emissions**—without sacrificing decision quality.

Why It Matters

Getting Goal Ingestion right is crucial because early inefficiencies have a compounding effect across the entire Agentic AI lifecycle. If this stage is handled poorly, it can lead to:

1. **Higher Costs:**
   Triggering unnecessary model calls, overloading context, or launching premature planning increases compute spend and cloud API usage unnecessarily.

2. **Higher Carbon Emissions:**
   Each avoidable compute cycle—whether on GPUs, CPUs, or cloud infrastructure—directly contributes to higher energy consumption and emissions.

3. **Reduced Scalability:**
   Systems that fail to manage resource use early become cost-prohibitive and operationally brittle as workloads grow.

A lean and green approach ensures that only well-defined goals and the essential context move forward, setting up every subsequent stage for success.

By embedding this discipline at the very first step, organizations not only reduce environmental impact but also build smarter, faster, and more cost-efficient AI systems that are scalable and sustainable by design.

Stage 2: Planning & Reasoning

The Planning & Reasoning stage is where an Agentic AI system determines how to fulfill the goal clarified in Stage 1. This is the system's equivalent of a project manager mapping out a sequence of actions, deciding on the right resources, and creating a plan that downstream agents will execute.

Given a clear objective, the system:

- Breaks down the goal into discrete subtasks,
- Decides what must be done, in what order, and
- Selects which agents, models, or tools will handle each step.

For example, in an ESG portfolio rebalancing scenario, the AI might produce a plan such as:

1. Retrieve current portfolio holdings,
2. Apply ESG screening to filter out low-scoring assets based on factors such as ESG ratings, news sentiment, financial results, and other relevant signals.

3. Perform risk/return optimization to select compliant assets,
4. Generate a recommendation report for the client.

The plan itself can be generated by:

- A rule-based workflow engine (deterministic),
- A small or medium-sized planning model, or
- A large general-purpose language model (LLM).

This leads to a crucial design choice in how planning is approached:
Open-Ended vs. Close-Ended Planning.

Open-Ended vs. Close-Ended Planning

A key decision in designing the Planning & Reasoning stage is choosing whether the system should follow an open-ended or close-ended approach to planning. The right choice directly impacts efficiency, cost, and environmental impact.

Open-Ended Planning:

- The system relies on **LLM-driven free-form reasoning** to figure out the sequence of steps dynamically.
- It is flexible, adaptable to **novel or less-defined problems**, and valuable when dealing with **highly contextual tasks**.
- However, it is:
    - More compute-intensive,
    - Slower due to multiple LLM passes,
    - Prone to retry loops or hallucinations, and
    - Higher in both cost and carbon footprint.

Open-ended planning works best for exploratory, one-off, or high-complexity tasks where no pre-defined pathway exists.

Close-Ended Planning:

- The system follows a pre-compiled or template-based plan for recurring, well-understood tasks.
- For example, an ESG portfolio rebalance might already have a standard playbook or decision tree stored and reusable.
- This is:
    - Extremely efficient,

  - Fast with low computational overhead, and
  - Environmentally friendly due to minimal resource use.

Close-ended planning is ideal for repetitive, stable processes where past knowledge can be re-applied safely.

Inefficiencies to Watch Out For

Poor planning decisions—whether open-ended or close-ended—can introduce multiple forms of **cost, computational waste, and emissions**:

1. Overuse of Open-Ended Planning:

- Using LLM-based free-form planning for every request, even for simple or routine tasks, leads to:
  - Excessive model calls,
  - Longer processing times,
  - Unnecessary emissions and cost.

Example: Generating the same type of ESG portfolio plan from scratch each time when a pre-defined template would have sufficed.

2. Retry Loops and Over-Planning:

- The agent may:
  - Call the LLM multiple times to refine the same plan,
  - Chase marginal gains in plan quality,
  - Fall into endless iteration without tangible improvement.

Each retry not only increases token consumption but adds to carbon emissions and infrastructure strain.

3. Agent Explosion:

- If planning is too granular or poorly scoped, the system may spawn unnecessary sub-agents—each with its own compute footprint and communication overhead.

Example: Separate agents for filtering ESG violations and ESG compliance scoring, when both could be integrated into a single efficient agent.

4. Prompt Bloat:

- Poorly designed prompts between planning agents (or between planners and executors) lead to:
  - Excessive token length,

  - Vague or redundant instructions, and
  - Increased LLM compute with no corresponding value.

Example:
Prompt: “Check the whole portfolio and advise on ESG issues and risks.”
Better: “For each holding, apply ESG Filter A. Flag assets below threshold. Output list with rationale.”

5. Over-Sized Model Usage:

- Invoking the largest available model for planning—regardless of task complexity—wastes compute cycles when smaller models, heuristics, or cached plans could suffice.

Lean and Green Principles Applied

To ensure that planning remains efficient, bounded, and sustainable, the following principles should be applied:

1. Choose the Right Planning Mode: Open-Ended vs. Close-Ended

- For novel, unique, or complex goals → Use Open-Ended Planning sparingly and with constraints.
- For recurring, predictable tasks → Default to Close-Ended Planning using pre-built workflows, templates, or cached strategies.

Building a hybrid approach—where the system can switch intelligently between open and close-ended modes—delivers both agility and efficiency.

2. Limit Planning Depth and Agent Creation

- Apply depth caps to prevent runaway task decomposition (e.g., no more than 3 layers deep).
- Set limits on parallel agent creation and encourage role consolidation when feasible.

3. Invoke Tools and Models Conditionally

- Only trigger data fetching, scoring models, or simulations if the plan actually requires it based on context.
  Example: Skip risk analysis if the portfolio change is minimal and within safe thresholds.

4. Use Structured, Minimal Prompts

- Design prompts using checklists, templates, and minimal context rather than free-form, verbose instructions.
- Every prompt should:

  - Communicate exactly what is needed,
  - Avoid open-ended phrasing,
  - Limit token size for cost and carbon savings.

5. Match Model Size to Planning Complexity

- Use small or mid-sized models for most routine planning.
- Reserve large models for:
  - Edge cases,
  - Highly novel goals,
  - Or when no pre-compiled knowledge exists.

If 90% of planning follows known pathways, cache and reuse instead of recomputing.

6. Control Retries and Reflections

- Set strict retry limits (e.g., max 2 attempts).
- If planning is still incomplete:
  - Fall back to a default plan,
  - Or escalate to human review.

7. Keep Context Lightweight

- During planning, the system should carry:
  - Only the Stage 1 goal,
  - Any recent outcome signals or constraints (e.g., previous failure reasons).
- Avoid pulling in full datasets, lengthy histories, or irrelevant memory at this stage.

Practical Illustration: Lean vs. Non-Lean Planning

Non-Lean Planning	Lean and Green Planning
Calls LLM five times chasing a “better” plan.	Caps retries at two; falls back to prebuilt plan or escalates.
Open-ended planning used for every request.	Chooses Close-Ended Planning for known patterns.

Spawns redundant agents for each subtask.	Consolidates functions; minimizes agent creation.
Sends verbose, wordy prompts with unnecessary context.	Uses concise, structured prompts with minimal tokens.
Uses the largest model for every plan.	Selects right-sized models based on task complexity.

Why It Matters

The Planning & Reasoning stage defines how efficient, scalable, and sustainable the entire Agentic AI system will be. Poor planning choices lead to:

- **Compute Overhead**: More models, more tokens, and more energy consumed unnecessarily.
- **Higher Latency**: Bloated plans lead to longer execution times.
- **Greater Carbon Emissions**: Each wasted cycle contributes to environmental impact.
- **Downstream Wast**e: Inefficient plans result in redundant computations, unnecessary tool calls, and poor resource utilization during execution.

By embedding lean and green planning principles, organizations can ensure that every plan is:

- Precise,
- Resource-aware, and
- Environmentally responsible.

Planning becomes not an exercise in endless ideation but a surgical blueprint for efficient action.

Stage 3: Tool & Model Selection

In Stage 3: Tool & Model Selection, the Agentic AI system decides precisely which models, tools, or services to use for each subtask identified in the plan.

If Stage 2 defines what needs to be done, this stage focuses on how to do it effectively—selecting the most appropriate, efficient, and sustainable resources for execution.

For example, if the plan calls for “calculate ESG scores for each portfolio holding,” the system may need to choose between:

- A small in-house ESG scoring model,

- An external ESG API, or
- A general-purpose language model capable of estimating scores from descriptions.

The decision considers multiple factors:

- Accuracy and suitability for the task,
- Cost of execution,
- Latency requirements,
- Carbon impact, and
- Availability of resources.

This selection process may involve dynamic loading of models or tools from a centralized repository or model catalog—an organized inventory of AI components, algorithms, APIs, and pre-trained models.

In multi-agent environments, modern systems increasingly adopt orchestration layers such as the Model Context Protocol (MCP) and Google's Agents-to-Agents (A2A) communication standard. These frameworks allow multiple agents to:

- Share context and tools,
- Avoid redundant model execution, and
- Access centralized repositories efficiently.

To ensure robustness, the system must also establish fallback mechanisms—preparing alternative tools or strategies in case the primary selection fails, without creating wasteful retry loops.

Inefficiencies to Watch Out For

Without a lean and green design, Tool & Model Selection can introduce several inefficiencies that escalate computational waste, costs, and emissions:

1. **One-Size-Fits-All Model Usage**

- Defaulting to large general-purpose models for every task—even simple ones—leads to disproportionate compute and energy use.

Example: Calling a 70B parameter model to perform basic classification or scoring when a smaller model would suffice.

2. **Redundant or Excessive Tool Invocation**

- Invoking multiple tools for the same task (e.g., dual ESG scoring to "double-check") consumes unnecessary compute cycles with no added value.

- Running sequential filters that could have been consolidated into one step further duplicates effort.

3. **Unnecessary Retries of Tools or Models**

- Systems may blindly retry failed tools or models, consuming compute and delaying results when smarter alternatives or graceful degradation would have been more efficient.

4. **Hardcoded, Rigid Choices**

- Always using the same model or resource—even when a more efficient alternative is available—wastes computational resources.
  Example: Calling GPU-intensive models when CPU inference would be faster and greener for smaller datasets.

5. **Over-Fetching Models and Tools**

- Pre-loading all available models, services, or datasets at the start of execution in anticipation of possible use consumes memory and compute, even if most of these resources are never actually utilized.

6. **Model Context Protocol (MCP) Overheads**

- In multi-agent orchestration environments where agents access models and tools via MCP and A2A, uncoordinated calls can lead to:
  - Network congestion,
  - Higher latency, and
  - Redundant execution of the same model across different agents.

Without centralized control, agents may each independently trigger the same expensive tool or model unnecessarily, increasing both cost and emissions.

7. **Tool Caching and Reuse Failures**

- Failing to cache expensive tool outputs—either locally or through a shared repository—results in avoidable recomputations.
- For example, if one agent computes ESG scores but doesn't cache them, another agent downstream may recompute the same scores—wasting resources that could have been saved through effective caching and retrieval.

Lean and Green Principles Applied

A Lean and Green Tool & Model Selection strategy focuses on smart, context-aware choices that minimize waste while maintaining task accuracy.

1. Use the Smallest Sufficient Model

- Apply right-sizing: select the smallest model that achieves acceptable performance.
- Larger models should be reserved for tasks that genuinely require complex reasoning or generation.

Example: Use a small ESG scoring model rather than a general-purpose LLM for simple compliance checks.

2. Prioritize and Streamline Tool Usage

- Avoid redundant tool selection; choose one best-fit tool for each task.
- Skip unnecessary steps (e.g., bypass heavy calculations if initial analysis indicates compliance).

3. Implement Carbon-Aware Scheduling

- Schedule energy-intensive operations to coincide with low-carbon grid availability—using carbon-intensity APIs to time execution strategically.

4. Apply Minimal Context for External Calls

- Send only essential information when calling external services or models—trimming inputs to avoid excessive token lengths or large data transfers.

5. Constrain Retries and Fallbacks

- Implement strict retry limits (e.g., one retry) and switch to fallbacks or graceful degradation strategies if failures persist.

6. Lazy Loading and Just-in-Time Tool Initialization

- Load or initialize models only when necessary—reducing idle energy consumption and memory overhead.

7. Centralized Repository and Shared Caching with MCP and A2A

- Host all models, tools, and algorithms in a centralized repository accessible by agents.
- Use MCP and A2A to enable shared context and caching, ensuring that:
    - Tool outputs are reused across agents,
    - Redundant calls are avoided, and
    - Network and compute use remain optimized.

Practical Illustration: Lean vs. Non-Lean Tool & Model Selection

Non-Lean Approach	Lean and Green Approach
Uses large general models by default.	Uses smallest fit-for-purpose models.
Calls multiple redundant tools for the same task.	Chooses one efficient tool; uses fallback only if needed.
Sends full datasets to remote services.	Sends minimal required inputs.
Pre-loads entire tool libraries "just in case."	Lazy loads tools on demand.
Retries tools indefinitely.	Caps retries and switches to fallback strategy.
Agents trigger redundant calls via MCP.	Centralized shared caching prevents duplication.

Why It Matters

Tool and Model Selection is one of the most high-impact levers for controlling both the operational and environmental footprint of Agentic AI systems. The choices made at this stage—what tools to use, what models to run, and how to orchestrate them—set the course for downstream efficiency or waste.

- **Cloud Costs:**
  Using an oversized general-purpose model (e.g., a 70B parameter LLM) when a small, specialized model could deliver the same result can multiply compute costs by 10x or more. Similarly, calling multiple redundant tools "just to be safe" can unnecessarily double or triple infrastructure charges.

- **Execution Latency:**
  Lean selection directly affects system responsiveness. Choosing a lighter model or a faster tool can cut response times from seconds to milliseconds, making AI outputs usable in real-time decision-making or high-frequency applications where speed is business-critical.

- **Carbon Emissions:**
  Larger models, unnecessary retries, or excessive tool calls consume more GPU or TPU time, which translates into higher energy use and emissions. For instance, running a large model for a task that could be handled by a small classifier could result in 5–20 times higher carbon emissions per action. Similarly, failure to cache results or coordinate via Model Context Protocol (MCP) leads to avoidable recomputation across agents, compounding energy waste.

By embedding lean selection principles—right-sizing models, streamlining tools, applying carbon-aware scheduling, and leveraging MCP and A2A orchestration with shared caching—organizations can:

- Lower infrastructure and API costs by cutting out redundant or oversized tools,
- Accelerate AI response times, improving user experience and decision-making,
- And reduce the carbon intensity of AI operations—without sacrificing intelligence, accuracy, or autonomy.

In short, Stage 3 is where precision in resource selection drives measurable gains across cost, speed, and sustainability. It is the point where Agentic AI can be made not just smarter, but also lighter, faster, and greener—by design

Stage 4: Action Execution

Action Execution is where the Agentic AI system carries out the plan using the selected tools, models, and data. This is the moment of delivery—where models are run, APIs are called, data is processed, and outputs are generated. The system fulfills its original goal through a coordinated set of operations.

In our ESG portfolio example, this is the phase where:

- Assets are scored for ESG compliance,
- Rebalancing logic determines new portfolio allocations, and
- A final recommendation or report is generated.

Execution may include:

- Local tasks (e.g., internal compute, rule-based scoring, on-device models),
- Remote tasks (e.g., sending prompts to LLM APIs, retrieving data from cloud services).

Often, this phase involves coordinating parallel workloads—such as real-time model execution, vector-based searches (e.g., RAG retrieval from an embedding store), or combined reporting tasks. The result is typically the final product—an answer, output, or completed action—ready for optional reflection or feedback handling in the next stage.

Inefficiencies to Watch Out For

This is the most compute-intensive phase, where inefficiencies translate directly into higher costs, longer runtimes, and elevated carbon emissions. Key pitfalls include:

1. **Excessive Remote Calls with Heavy Payloads**

- Overuse of external services with large, verbose prompts (as seen in RAG scenarios) leads to inflated token costs, compute demand, and latency.

2. **Idle or Oversized Infrastructure**

- Running workloads on over-provisioned, always-on servers—such as multi-GPU machines or large VMs—for sporadic tasks leads to energy waste.

- Similarly, keeping heavyweight services running when not in use burns unnecessary electricity.

3. **Overly Verbose Outputs**

- Generating long, unnecessary narratives when concise outputs would suffice drives excess model calls, token generation, and post-processing.

4. **Inefficient Data Handling**

- Pulling full datasets when only subsets are needed or having agents process identical data independently increases memory use and compute cycles.

5. **Lack of Batching**

- Running one-off computations sequentially rather than processing in optimized batches underutilizes compute resources.

6. **Redundant Computations**

- Agents recomputing the same values or duplicating effort because of poor inter-agent coordination or missing shared memory.

7. **Multiple Passes for Similar Output**

- Generating both detailed reports and separate summaries unnecessarily when one output could have sufficed with proper design.

8. **Suboptimal Model and Storage Setup**

- Using oversized models or non-optimized storage systems (e.g., always-on large vector databases) for simple tasks creates hidden inefficiencies.

- Storing intermediate results without compression, deduplication, or tiered storage increases energy consumption and storage costs.

- Failing to match model hosting infrastructure (CPU vs. GPU vs. serverless) to task complexity wastes both time and energy.

Lean and Green Principles Applied

A Lean Action Execution strategy focuses on making every action, model run, and data movement efficient, purposeful, and sustainable. Below are the key principles, along with practical examples and the rationale behind each.

1. Optimize Remote Execution

Compress prompts and minimize retrieved context when using external services such as RAG. For instance, instead of sending entire datasets to an LLM, send only key summaries or decision-relevant highlights.This reduces token count, speeds up

responses, and lowers both compute cost and emissions. Where possible, cache the results of external calls to avoid repeated requests for similar queries.

2. Use Right-Sized, Efficient Infrastructure

Choose compute environments that align with the actual workload:

- Simple classification tasks can run on CPUs or small instances.
- Heavy inference tasks can use GPUs—but only when needed, and with auto-scaling.
- Stateless operations can be offloaded to serverless functions that scale to zero when idle.

For example, portfolio optimization jobs can run as short-lived serverless functions instead of tying up large, always-on GPU VMs. This reduces idle power draw, operational cost, and wasted emissions.

3. Optimize Model Setup

Select models that are as lightweight as possible for the task. Use distilled, quantized, or compressed models for routine jobs where ultra-high accuracy is not mission-critical. Store frequently used models in optimized containers or fast-access storage to cut down on cold start times and resource waste.

For example, using a small BERT-like model for document classification instead of defaulting to a heavyweight generative model can deliver faster results at a fraction of the energy and cost.

4. Limit Output Verbosity

Instruct AI systems to generate **concise, structured outputs** by default. Instead of producing lengthy narratives, encourage outputs like:

- Bullet-point summaries,
- Executive highlights,
- Short form responses capped by word or token count.

For instance, guiding an LLM to respond with “3 key points in under 100 words” not only cuts down on processing time but directly reduces API costs (which are often token-based) and speeds up end-user consumption.

5. Carbon-Aware Scheduling

For tasks that don’t require immediate execution—such as batch processing, model retraining, or report generation—schedule workloads during times when the electricity grid is greener.

For example, scheduling heavy computations overnight when wind power is abundant or mid-day when solar peaks can significantly lower the emissions per task without delaying business outcomes.

6. Minimize Data Movement and Storage Waste

Process only the data that's essential for the current task:

- Apply early filtering,
- Use streaming rather than bulk loading,
- Store intermediate results using compressed formats or place less-used data in lower-cost storage tiers.

For example, instead of exporting full datasets for analysis, extract and process only the necessary records or dimensions.

7. Batch Operations for Efficiency

Group repetitive tasks into batches to take full advantage of hardware acceleration. For example, scoring 1000 assets for ESG compliance can be processed in 10 batches of 100 rather than as 1000 individual calls.
Batching reduces overhead, speeds up computation, and minimizes per-task energy usage.

8. Eliminate Duplicate Work

Design execution workflows that reuse already computed results wherever possible:

- Share intermediate outputs across agents,
- Implement result caching to prevent recalculations.

For example, if an agent has already generated ESG scores for a set of assets, that result can be shared rather than recalculated by multiple downstream components.

Why It Matters

Action Execution is where computational decisions directly translate into operational costs, performance outcomes, and environmental impact. Optimizing this stage delivers measurable benefits across multiple dimensions:

- **Operational Cost:**
  Generating overly verbose outputs or using oversized models for simple tasks can increase cloud costs by 2 to 10 times for the same outcome. Similarly, running workloads on over-provisioned servers when smaller, on-demand environments would suffice leads to unnecessary infrastructure spend.

- **Execution Speed:**
  Sequentially running small tasks without batching can make execution five times slower than using optimized, parallel processing. This not only delays responses but reduces system throughput and efficiency.

- **Carbon Impact:**
  Running the same workload on idle, always-on GPU instances instead of using serverless or auto-scaled compute can significantly increase energy consumption and emissions. Scheduling tasks during high-carbon grid periods further compounds the environmental impact.

By embedding lean execution principles, organizations can:

- Deliver faster, more cost-effective AI outcomes,
- Minimize carbon footprint,
- And build systems that are resilient, scalable, and sustainable by design.

Stage 5: Reflection & Retry

The Reflection & Retry stage is the final decision point in the Agentic AI lifecycle—where the system doesn't just execute but steps back to assess:

- Has the goal been achieved?
- Is the outcome aligned with constraints and quality expectations?
- Should the result be accepted, adjusted, or escalated?

This is the moment where the agent behaves not only as a doer, but also as a reviewer and decision-maker.

In practice, this may involve:

- Checking if numeric thresholds (such as ESG or risk scores) are met,
- Evaluating the clarity or completeness of an output,
- Deciding whether to retry a specific step or finalize and move forward.

Reflection may happen automatically or in response to human feedback—for instance, when a user flags that an answer isn't satisfactory.

Inefficiencies to Watch Out For

Reflection adds value, but without careful design, it can become a hidden source of compute waste, cost, and emissions.

1. **Unbounded Retry Loops**
   Without clear stopping conditions, agents may repeatedly retry or refine results—consuming unnecessary compute for diminishing returns.

2. **Heavy Models for Light Checks**
   Using large models for minor corrections or validations leads to disproportionate resource consumption.

3. **Redundant Prompting**
   Re-asking the same question with minimal variation leads to repeated processing without outcome improvement.

4. **Over-Reflection on Acceptable Results**
   Chasing perfection when results already meet predefined success criteria causes unnecessary iterations.

5. **Lack of Success Criteria or Escalation**
   Without clear definitions of “done” or when to escalate to a human, systems risk over-automation or ineffective retries.

Lean and Green Principles Applied

A Lean Reflection & Retry strategy ensures quality without unnecessary repetition, energy use, or delays.

1. **Set Clear Retry Limits and Success Thresholds**
   - Apply caps on retries (e.g., two attempts) and define clear criteria for what constitutes a satisfactory outcome.
   - This prevents infinite loops and keeps compute use predictable.

2. **Ensure Meaningful Changes on Retry**
   - Each retry should introduce a specific adjustment: changing a prompt, using a different model, or applying a new parameter.
   - Avoid redundant retries that repeat the same inputs expecting different outcomes.

3. **Use Lightweight Reflection by Default**
   - Where possible, use basic logic or small models for checks before escalating to more resource-intensive models.
   - For example, use numerical checks for constraints instead of querying an LLM.

4. **Minimize Context in Retried Calls**
   - Do not resend full datasets or prompts unnecessarily. Focus retries on the specific area requiring correction.

5. **Monitor and Learn from Reflection Patterns**
   - Track the frequency, causes, and costs of retries.
   - Use this insight to continuously improve earlier stages and reduce the need for future reflection.

6. **Incorporate Human Feedback and Confidence Thresholds**
    - Allow human review or confidence scoring to guide when reflection is needed versus when it can be safely skipped.
7. **Apply Sustainability-Aware Retry Decisions**
    - Before triggering retries, consider the potential value of the improvement relative to the cost, timing, and environmental impact of running an additional cycle.

Why It Matters

Optimizing Reflection & Retry prevents this phase from becoming an invisible cost driver:

- **Operational Cost:**
  Bounded retries and lightweight validation methods help avoid unnecessary compute spend and runaway infrastructure costs.
- **Execution Speed:**
  Lean reflection delivers timely results without unnecessary delays, keeping AI systems responsive and efficient.
- **Carbon Impact:**
  Minimizing unnecessary retries, especially when heavy models are involved, reduces energy use and emissions, making the entire AI process more sustainable.

A well-designed Reflection & Retry stage acts as a smart, sustainable control point, balancing quality assurance with efficiency—ensuring that AI systems deliver value without waste.

Stage 6: Memory & Learning

The Memory & Learning stage allows the Agentic AI system to retain valuable insights and preferences from past tasks to improve future performance. Memory enables the AI to avoid starting from scratch every time, enhancing efficiency, personalization, and decision quality over repeated interactions.

In practice, this includes:

- Storing final outcomes (e.g., approved portfolio recommendations),
- Capturing user-specific preferences and constraints,
- Recording what worked and what failed for future learning.

Memory can be:

- Session-specific (temporary),

- Shared across agents (collaborative),
- Persistent across time (long-term learning).

Inefficiencies to Watch Out For

Without careful design, memory management can create both performance drag and sustainability concerns:

1. **Memory Bloat:**
   Saving every detail (logs, prompts, intermediate results) clutters the system and leads to slower retrieval and higher storage costs.

2. **Duplicate Storage:**
   Multiple agents may redundantly save the same data, wasting storage and creating inconsistency risks.

3. **Unfiltered Learning:**
   Storing both successful and failed outcomes without context risks future errors and unnecessary clutter.

4. **Inefficient Retrieval:**
   Unoptimized memory can make lookups computationally expensive, especially when using large vector stores or injecting excessive memory into prompts.

5. **Context Overload:**
   Including too much irrelevant memory in decision-making or LLM prompts increases token cost, slows down response times, and can confuse model outputs.

6. **Persistent Storage and Energy Waste:**
   Retaining unnecessary long-term data consumes ongoing storage energy—even when the information is no longer useful.

Lean and Green Principles Applied

A **Lean Memory & Learning strategy** ensures that what is stored, retrieved, and used adds value without overloading the system or wasting resources.

1. Store High-Value Information Only

- Save only what is actionable and impactful: final decisions, key preferences, and rationale—not every raw step or failed attempt.
- Example: Store “Client 84721 prefers no coal investments, approved portfolio X”—not full process logs.

2. Surgical Memory Retrieval

- Retrieve only the relevant memory for each new task, using structured storage or targeted similarity search.

- Example: In RAG scenarios, limit retrieval to the most relevant few results rather than pulling large volumes of context.

3. Inject Only What's Needed into Context

- When using memory to inform AI prompts or decision flows, include only concise, relevant snippets—not entire histories or irrelevant past details.
- Example: Pulling just the previous ESG preference, not the full dialogue history.

4. Prune and Expire Memory

- Implement retention policies: discard low-value or stale information after a set time or after summarization.
- Example: Keep detailed logs for 30 days, then archive only key summaries for long-term use based on application's data severity requirements .

5. Structure Memory for Efficiency

- Organize memory using simple structures (key-value pairs, tagged data) that support fast, direct retrieval without complex searches.

6. Exclude Known Bad or Irrelevant Information

- Mark failed strategies or outdated data as excluded from future planning to avoid repeating mistakes or carrying unnecessary baggage.

Why It Matters

Memory, when managed well, is the engine of continuous improvement—but when unmanaged, it becomes a source of waste.

- **Faster Learning Curve:**
  By focusing on key takeaways, agents improve without unnecessary rework—leading to faster, more accurate results.
- **Lower Resource Use:**
  Efficient memory reduces storage costs, energy consumption, and the computational overhead of retrieval and context injection.
- **Better Output Quality:**
  Selective memory ensures that models receive relevant, high-quality context, improving decision-making and reducing retries.
- **Sustainability:**
  Avoiding digital hoarding reduces both storage emissions and future processing waste, aligning AI with environmental goals.

By embedding Lean Memory & Learning, Agentic AI systems grow smarter while staying efficient, scalable, and environmentally responsible.

Supporting Practices for Lean and Green AI

While the six lifecycle stages of Agentic AI define where optimization happens, these supporting practices strengthen the technical foundation for achieving energy efficiency, cost reduction, and emissions minimization across the entire AI system.

1. Energy-Efficient Model Architectures

Designing or selecting models that require fewer computations for equivalent output is one of the most direct ways to cut energy use.

- Techniques like pruning (removing unnecessary weights) and quantization (using lower-precision numbers) significantly reduce compute demand.
- Efficient architectures such as sparse transformers or hybrid models can deliver strong performance with fewer floating point operations (FLOPs).
- Example: A quantized model running on edge devices may cut inference energy by 50% while maintaining accuracy within 1%.

Smaller, well-optimized models not only consume less power but also offer faster inference and lower cloud costs, making them ideal for both sustainability and performance.

2. Optimized Training Processes

Training is often the most energy-intensive stage in an AI model's lifecycle. Applying lean principles here yields substantial impact.

- Transfer learning reduces the need for full-model training by reusing pre-trained backbones.
- Mixed-precision training (e.g., FP16, BF16) accelerates training while reducing power use with minimal accuracy loss.
- Carbon-aware training schedules align training runs with periods of lower grid emissions.
- In some cases, it's acceptable to trade off a slight drop in accuracy for significantly lower training compute—especially for non-critical tasks.

Example: Fine-tuning a language model with mixed-precision and fewer epochs can reduce training emissions by up to 60% without impacting end-user experience.

3. Efficient Inference Techniques

Once deployed, models often serve millions of inferences—making inference efficiency essential.

- Model distillation produces smaller student models that replicate larger models' behavior while consuming far less compute per prediction.
- Using accelerated hardware (e.g., TPUs, AI-optimized GPUs) increases operations-per-watt compared to general-purpose CPUs.
- Applying prompt engineering techniques—such as using concise, well-scoped prompts or prompt templates—reduces input and output token lengths, directly lowering inference time, compute load, and energy consumption.
- Batching and caching results for repetitive queries maximizes hardware utilization and reduces redundant computation.

Example: A distilled classifier model deployed on edge TPUs may respond five times faster and cut power usage by 70% compared to the original large model on CPU.

4. Carbon-Aware Execution

Aligning when and where AI workloads run with the availability of low-carbon energy significantly reduces emissions without altering the task outcome.

- Use carbon intensity APIs from cloud providers or grid operators to schedule non-urgent workloads (e.g., batch processing, retraining) during cleaner energy windows.
- Shift computation geographically—for example, from a region with coal-heavy energy to one with high wind or solar input.
- Organizations running private infrastructure can integrate on-site renewables or carbon-aware job orchestration.

Example: Running model training at night in a wind-powered data center versus midday in a coal-heavy region can reduce carbon emissions from that workload by over 60%.

Why Supporting Practices Matter

These technical practices enhance and complement lifecycle-stage optimizations:

- **Energy-Efficient Models** reduce the footprint of each model call—every prediction is cheaper and greener.
- **Training Optimizations** avoid redundant work and lower the carbon cost of model development.
- **Efficient Inference** ensures scale doesn't lead to runaway resource usage.
- **Carbon-Aware Scheduling** makes smart use of external environmental conditions to reduce total emissions.

While each practice involves trade-offs—such as sacrificing slight accuracy or modifying SLAs—they often lead to side benefits: faster responses, lower hosting costs, and improved hardware efficiency.

Together, these practices enable organizations to green the full AI stack—from model design and data movement to hardware use and power grid alignment. A truly Lean and Green Agentic AI system is one that optimizes not only task-by-task but also strategically across the entire compute lifecycle.

Measuring Carbon Impact: The Software Carbon Intensity (SCI) ISO Standard

One of the central challenges in making AI—and software in general—more sustainable is the lack of consistent, actionable measurement. You can't improve what you don't measure. The Software Carbon Intensity (SCI) Framework, now formalized as ISO/IEC 21031:2024, provides a standardized, repeatable way to measure the carbon emissions attributable to software systems, including AI workloads.

Developed by the Green Software Foundation, SCI helps organizations answer a simple but crucial question:
"How much carbon does it take to perform a given software function?"

For AI, this could mean:

- Grams of CO_2 per 1000 inference calls,
- Kilograms of CO_2 per training job,
- Grams of CO_2 per user transaction or prompt/tokens.

By quantifying emissions per unit of work, organizations can track, compare, and reduce the environmental impact of AI systems with the same rigor they apply to accuracy, cost, and speed.

How SCI Works: Five Practical Steps

1. Bound the System

Define what part of the system you're measuring. This could include:

- Model inference only,
- Training pipeline,
- Or the entire end-to-end lifecycle of an AI agent.

Example: Measuring the SCI of a portfolio rebalancing AI service including data retrieval, model execution, and reporting—but excluding user device energy.

A clear boundary ensures transparency and comparability.

2. Define the Functional Unit (Scale)

Choose the appropriate unit of work to normalize the measurement:

- Per transaction, per inference, per training run, or per hour of service.

Example: For an inference service, you might choose “per 1000 predictions” as the unit.

This allows meaningful comparisons over time or between systems.

3. Select a Measurement Method

Determine how energy use will be calculated:

- **Direct telemetry** (actual power usage from infrastructure),
- **Estimation models** (cloud provider data, hardware specs, utilization rates),
- External carbon tracking tools or APIs.

Transparent documentation of the chosen method is key for accuracy and trust.

4. Quantify the SCI Score

Calculate the SCI by dividing:
Total Carbon Emissions ($kgCO_2$) ÷ Total Functional Units

Example:
If running 1000 inferences consumes 0.5 kWh, and grid carbon intensity is 0.4 $kgCO_2$/kWh:

- Total emissions = 0.5 × 0.4 = 0.2 $kgCO_2$.
- SCI = 0.2 $kgCO_2$ per 1000 inferences, or 0.2 grams CO_2 per inference.

5. Report and Track

Share the SCI score with context:

- The scope measured,
- The functional unit used,
- The measurement approach applied.

Example:
“Our AI service emits 5 grams CO_2 per transaction (measured August 2025, US-West) following lean optimizations—down from 8 grams earlier this year.”

Regular reporting drives accountability, continuous improvement, and industry benchmarking.

SCI for AI: Extending the Standard to AI Workloads

The Green Software Foundation (GSF) is actively working to extend the ISO SCI standard specifically for AI systems through the development of SCI for AI. This extension builds upon the core principles of the Software Carbon Intensity (SCI)

specification but expands them to address the distinct characteristics of AI workloads.

SCI for AI focuses on:

- The unique architectural patterns of AI systems, including model-centric and agent-centric designs,
- The multi-stage operational lifecycle of AI—spanning training, fine-tuning, inference, and agent orchestration,
- The need for clear boundary definitions in AI systems, where AI components often interact with multiple services, models, and infrastructures,
- The importance of defining functional units that make AI emissions comparable, measurable, and actionable—for example, emissions per inference, per token, or per agent task.

SCI for AI aims to ensure that even complex AI workflows can be measured with consistency, transparency, and comparability. This work is progressing toward ISO standardization.

For the latest developments, see the official **SCI for AI workshop report**: https://greensoftware.foundation/articles/sci-for-ai-workshop-report

Why It Matters

By embedding SCI into AI development and operations, organizations can:

- **Track Measurable Progress:**
  Quantify the impact of Lean and Green optimizations over time—moving from assumptions to verifiable results.
- **Compare Design Choices:**
  Evaluate the environmental cost of different models, tools, or deployment strategies—helping teams select not just the fastest or cheapest option, but also the **cleanest**.
- **Drive Sustainability Goals:**
  Integrate SCI metrics into corporate carbon reduction targets, procurement policies, and public reporting.
- **Enable Continuous Optimization:**
  SCI creates a cycle of **measure → improve → verify**, ensuring that sustainability becomes part of the everyday decision-making process.

In short, **SCI moves sustainability from anecdote to evidence**—providing a foundation for balanced decisions where accuracy, speed, cost, and carbon are all weighed together. For AI systems, this is not just an environmental necessity—it's becoming a business expectation.

Summary

The shift to Lean and Green Agentic AI is not just a technical evolution—it is a strategic and economic imperative.

As AI systems grow in complexity, scale, and autonomy, so too does their computational footprint, operational cost, and environmental impact. Without deliberate design, agentic architectures risk driving up energy consumption, carbon emissions, and cloud costs—all while increasing system complexity.

This white paper has shown that sustainability and cost-efficiency are not constraints—they are enablers of scalable, intelligent, and commercially viable systems.

By applying lean principles across the six lifecycle stages—from Goal Ingestion to Memory & Learning—and reinforcing them with supporting efficiency practices and SCI-based carbon measurement, organizations can:

- Reduce energy consumption, emissions, and operational costs without sacrificing intelligence or autonomy,
- Improve system responsiveness and scalability while lowering the total cost of ownership,
- Create accountability through transparent, measurable metrics such as the Software Carbon Intensity (SCI) score,
- Align AI development with enterprise sustainability targets and financial objectives.

This approach enables a new class of AI—agentic, adaptive, cost-effective, and environmentally responsible by design.

The path forward is clear:

- Embed efficiency at the architecture level,
- Monitor both carbon and cost using actionable metrics,
- Iterate continuously to balance cost, performance, accuracy, and environmental impact,
- And scale AI systems responsibly and economically.

Sustainable AI isn't just possible—it's measurable, cost-saving, and increasingly non-negotiable. The next generation of agentic systems must not only think and act—but do so leaner, faster, greener, and cheaper.

References

For further reading on how to design efficient, sustainable, and scalable agentic AI systems, refer to:
Lean Agentic AI: Minimizing Cost, Carbon, and Complexity
https://leanagenticai.com/

This is the first book of its kind: a practical, actionable guide to building agentic AI systems that not only deliver intelligence but do so with minimal cost, carbon, and complexity.

Disclaimer: The content of this white paper is based on the concepts introduced in the book **Lean Agentic AI**. It is intended for general informational purposes and is not tied to any specific product, framework, or vendor. This is a personal publication and does not represent the views of any organization or employer.

About the Author

Navveen Balani is an entrepreneur, researcher, and innovator with a proven track record in business and exponential technology transformation. With over two decades of experience, he specializes in crafting state-of-the-art solutions that drive business growth. Navveen is a Google Certified Cloud Fellow, a LinkedIn Top Voice in AI, and the author of numerous leading technology books, including *Lean Agentic AI: Minimizing Cost, Carbon, and Complexity*.

As Chair of the Standards Working Group at the Green Software Foundation, he is helping lead the development of the Software Carbon Intensity (SCI) for AI specification—bringing industry-wide standards to the measurement and reduction of AI carbon emissions.

Navveen is passionate about raising awareness on responsible, sustainable, and practical AI through his books, blogs, and YouTube channel, making complex topics accessible and actionable for organizations and individuals alike.

You can reach him on LinkedIn or visit www.navveenbalani.dev for more information.